Approximation and String Algorithms

Thomas Nowak

These are the lecture notes for the M1 course Approximation and String Algorithms at ENS Paris-Saclay in the academic year 2026–2027. Good references for the material covered here include Williamson and Shmoys [1] for approximation algorithms and Lothaire [26] for the combinatorics on words underlying the string algorithms. The website for the course is at `https://algo.biodis.co/`.

This document will inevitably contain some errors. If you find any, I will be grateful and happy to hear from you. You can reach me by email at thomas@thomasnowak.net.

Thomas Nowak
ENS Paris-Saclay
Academic Year 2026–2027
Last Update: September 2, 2026

Contents

September 7, 2026

Lecture 1: Approximation Algorithms I

Many fundamental optimization problems are NP-hard, meaning that finding exact optimal solutions is computationally infeasible for large instances. Approximation algorithms provide a principled approach to finding near-optimal solutions efficiently, with provable guarantees on solution quality. The key insight is that for many practical problems, we don't actually need the perfect solution: we just need a solution that's provably close to optimal and can be computed quickly.

1.1 Foundations and Definitions

An optimization problem asks us to find a feasible solution that minimizes (or maximizes) some objective function. For a minimization problem, let OPT denote the optimal solution value and ALG the value produced by algorithm A.

Definition 1.1

Algorithm A is an α-approximation algorithm if

$$\text{ALG} \leq \alpha \cdot \text{OPT}$$

for all instances of the problem, where $\alpha \geq 1$ is called the approximation ratio.

For maximization problems, we require $\text{ALG} \geq \text{OPT}/\alpha$. The closer α is to 1, the better the approximation guarantee. An approximation ratio of $\alpha = 2$ means our solution is at most twice as expensive as optimal, which is not perfect but often good enough in practice.

The landscape of approximable problems is characterized by several complexity classes. The class APX contains problems that admit constant-factor approximation algorithms. These are the "nice" NP-hard problems: hard to solve exactly, but not impossibly hard to approximate. A Polynomial-Time Approximation Scheme (PTAS) is more refined: for any $\varepsilon > 0$, there exists a $(1+\varepsilon)$-approximation algorithm running in time polynomial in n (though possibly exponential in $1/\varepsilon$). For example, running time $O(n^{1/\varepsilon})$ is allowed, which becomes impractical for small ε. The strongest guarantee is a Fully Polynomial-Time Approximation Scheme (FPTAS), which runs in time polynomial in both n and $1/\varepsilon$, such as $O(n^2/\varepsilon)$.

Not all NP-hard problems admit good approximations. Some problems are provably hard to approximate within any reasonable factor unless P = NP, while others admit PTAS or even FPTAS. Understanding which problems fall into which category is a central question in approximation algorithms.

1.2 Vertex Cover

The vertex cover problem serves as our first case study. Given an undirected graph $G = (V, E)$, a vertex cover is a subset $C \subseteq V$ such that every edge has at least one endpoint in C. We want to find a vertex cover of minimum size.

A natural greedy approach is to repeatedly select the vertex of maximum degree, add it to the cover, and remove all incident edges. Let us analyze both the performance and limitations of this approach.

Theorem 1.1. The greedy maximum degree algorithm achieves an approximation ratio of $O(\log n)$ for vertex cover.

Proof. Let C be the cover produced by the greedy algorithm. We will track the number of edges remaining after each iteration. Let $E_0 = E$ and let E_i denote the set of edges remaining after the i-th iteration, when we have selected i vertices for the cover.

In iteration $i+1$, we select a vertex v of maximum degree Δ_i in the graph (V, E_i). This vertex covers Δ_i edges, so $|E_{i+1}| = |E_i| - \Delta_i$.

Now consider the optimal solution, which uses at most OPT vertices to cover all edges. In particular, these OPT vertices must cover the $|E_i|$ edges that remain in E_i. By the pigeonhole principle, at least one vertex in the optimal solution must cover at least $|E_i|/\mathrm{OPT}$ of these edges.

Since the greedy algorithm selects the vertex of maximum degree, we have:

$$\Delta_i \geq \frac{|E_i|}{\mathrm{OPT}}$$

Therefore:

$$|E_{i+1}| = |E_i| - \Delta_i \leq |E_i| - \frac{|E_i|}{\mathrm{OPT}} = |E_i| \left(1 - \frac{1}{\mathrm{OPT}}\right)$$

Unrolling this recurrence from $i = 0$ to $i = t-1$ where t is the number of iterations:

$$|E_t| \leq |E_0| \left(1 - \frac{1}{\mathrm{OPT}}\right)^t$$

The algorithm terminates when $|E_t| = 0$, so we need:

$$|E_0| \left(1 - \frac{1}{\mathrm{OPT}}\right)^t < 1$$

Using the inequality $(1-x) \leq e^{-x}$ for $x > 0$:

$$|E_0| \left(1 - \frac{1}{\mathrm{OPT}}\right)^t \leq |E_0|\, e^{-t/\mathrm{OPT}}$$

The right-hand side is less than 1 when $t > \mathrm{OPT} \cdot \ln|E_0|$. Therefore the algorithm terminates with at most $\lceil \mathrm{OPT} \cdot \ln|E_0| \rceil$ vertices. Since $|E_0| \leq \binom{n}{2} < n^2$, we have $|C| \leq \mathrm{OPT} \cdot \ln n^2 + 1 = O(\mathrm{OPT} \cdot \log n)$. □

This seems promising: a logarithmic approximation ratio. However, the greedy approach can perform poorly on certain graph families.

Theorem 1.2. There exists a family of graphs on which the greedy maximum degree algorithm produces a cover of size $\Omega(\mathrm{OPT} \cdot \log n)$.

Proof. Consider the family of bipartite graphs $G_r = (L \cup R, E)$ constructed as follows. Let $L = \{\ell_1, \ldots, \ell_r\}$ and partition $R = R_1 \cup R_2 \cup \cdots \cup R_r$ where $|R_i| = \lfloor r/i \rfloor$. We connect each vertex in R_i to exactly i distinct vertices in L, subject to the constraint that every vertex in L has at most one neighbor in R_i. This is possible because R_i needs $i \cdot \lfloor r/i \rfloor \leq r$ distinct endpoints in L.

Concretely, label the vertices in R_i as $v_1^{(i)}, \ldots, v_{\lfloor r/i \rfloor}^{(i)}$ and connect $v_j^{(i)}$ to the i vertices $\ell_{(j-1)i+1}, \ell_{(j-1)i+2}, \ldots, \ell_{ji}$. This way, within each group R_i, the neighborhoods partition (a subset of) L into blocks of size i, so no vertex in L has more than one neighbor in R_i. As a result, every vertex in R_i has degree exactly i, and every vertex $\ell_j \in L$ has degree $|\{i : 1 \leq i \leq r,\ \ell_j \text{ has a neighbor in } R_i\}|$. See Figure 1.1 for an

illustration with $r = 4$.

The optimal solution is $\mathrm{OPT} = r$ (take all vertices in L, which covers every edge). However, the greedy algorithm can be forced to select all vertices in R. The maximum degree in G_r is r, achieved by the single vertex in R_r, so greedy selects it first. After removing its r incident edges, the remaining graph has maximum degree $r-1$ among the vertices in R_{r-1}, so greedy continues by picking vertices from R_{r-1}, then R_{r-2}, and so on down to R_1.

The total number of vertices selected is:

$$|R| = \sum_{i=1}^{r} \lfloor r/i \rfloor \approx r \sum_{i=1}^{r} \frac{1}{i} = r \cdot H_r \approx r \log r = \Omega(\mathrm{OPT} \cdot \log r)$$

□

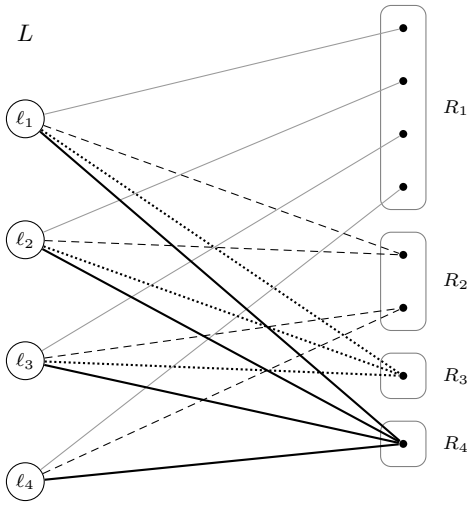

Figure 1.1: The graph G_4 with $r = 4$. Each vertex in R_i has degree exactly i. Edge styles distinguish groups: R_1 thin gray, R_2 dashed, R_3 dotted, R_4 thick solid.

This shows the greedy algorithm's approximation ratio is tight at $\Theta(\log n)$.

Can we achieve a *constant-factor* approximation? Rather than guessing an algorithm and analyzing it after the fact, let us think about what a proof of a c-approximation would have to look like, and let the proof structure guide us to an algorithm.

Suppose an algorithm produces a cover C and we claim $|C| \leq c \cdot \mathrm{OPT}$. Since computing OPT is NP-hard, the proof cannot simply compare $|C|$ to the optimal value. Instead, the proof must exhibit some efficiently verifiable *certificate* that OPT is large enough, that is, that $\mathrm{OPT} \geq |C|/c$.

What kind of certificate can witness that every vertex cover is large? Each edge imposes a *demand*: at least one of its endpoints must be in the cover. A single edge certifies $\mathrm{OPT} \geq 1$; two edges certify $\mathrm{OPT} \geq 2$—but only if they don't share an endpoint, since otherwise a single vertex could satisfy both demands simultaneously. More generally, to certify $\mathrm{OPT} \geq k$, we need k edges whose demands are independent, meaning no two of them can be satisfied by the same vertex. This happens exactly when the edges are *pairwise disjoint*: they share no endpoints.

So a c-approximation proof needs a set M of pairwise disjoint edges with $|C| \leq c \cdot |M|$. What if the algorithm itself builds M as a byproduct? The simplest idea: take both endpoints of every edge in M as the cover, giving $|C| = 2|M|$ and thus $c = 2$. But is this actually a valid vertex cover? An edge $e \notin M$

could escape coverage if neither of its endpoints belongs to C. This cannot happen if M is *maximal*: if every edge in E shares an endpoint with some edge of M, then every edge is covered. This leads us to the following algorithm.

Algorithm:

1. Initialize $C = \emptyset$ and $E' = E$.
2. While $E' \neq \emptyset$:
   (a) Pick any edge $\{u, v\} \in E'$.
   (b) Add both u and v to C.
   (c) Remove all edges incident to u or v from E'.
3. Return C.

Note that this procedure computes a maximal matching M (the set of picked edges) and returns both endpoints of every edge in M.

Theorem 1.3. The edge-picking algorithm is a 2-approximation for vertex cover.

Proof. Let M denote the set of edges selected by the algorithm. By construction, M is a maximal matching: edges in M are pairwise disjoint, and no remaining edge can be added because all edges incident to selected vertices have been removed. Since we add both endpoints of each selected edge, $|C| = 2|M|$.

By maximality, every edge in E is incident to at least one vertex in C, so C is a valid vertex cover. Since the edges in M are pairwise disjoint, any vertex cover must include at least one distinct endpoint per edge, so $\mathrm{OPT} \geq |M|$. Therefore:

$$|C| = 2|M| \leq 2 \cdot \mathrm{OPT}$$

□

The 2-approximation for vertex cover has been known since the 1970s, and despite decades of effort, no polynomial-time algorithm with a better constant factor has been found.

It is natural to ask whether we can do better than the 2-approximation. This turns out to be a deep question connected to fundamental conjectures in complexity theory.

On the hardness side, Dinur and Safra [2] proved that it is NP-hard to approximate vertex cover within a factor of 1.3606. This means that unless P = NP, no polynomial-time algorithm can guarantee a solution strictly better than $1.3606 \cdot \mathrm{OPT}$.

Even stronger hardness results are known under the Unique Games Conjecture (UGC), introduced by Khot [4]. The UGC posits that a certain type of constraint satisfaction problem, where each constraint uniquely determines one variable given the other, is hard to approximate. Under this assumption, Khot and Regev [5] showed that vertex cover cannot be approximated within any factor $2 - \varepsilon$ for constant $\varepsilon > 0$.

If the UGC is true, then our simple 2-approximation algorithm is essentially optimal. The gap between the 2-approximation upper bound and the current best hardness lower bound of 1.3606 (without assuming UGC) remains one of the major open problems in approximation algorithms.

1.3 Set Cover

The set cover problem generalizes vertex cover and many other covering problems. We are given a universe U with $|U| = n$ and a collection of subsets $\mathcal{S} = \{S_1, S_2, \ldots, S_m\}$ where $S_i \subseteq U$. The goal is to find a minimum cardinality subcollection $\mathcal{C} \subseteq \mathcal{S}$ such that the selected sets cover all elements, that is, $\bigcup_{S \in \mathcal{C}} S = U$.

Lemma 1.1. Vertex cover is a special case of set cover.

Proof. Consider a graph $G = (V, E)$. Let $U = E$ be the universe (the edges), and for each vertex $v \in V$, let $S_v = \{e \in E : v \in e\}$ be the set of edges incident to v. Then a vertex cover corresponds exactly to a set cover: we select vertices (sets) such that every edge (element) is covered.

Formally, $C \subseteq V$ is a vertex cover if and only if $\{S_v : v \in C\}$ is a set cover of $U = E$. Since $|C| = |\{S_v : v \in C\}|$, the optimal solutions have the same size. □

The natural greedy algorithm makes the locally optimal choice at each step: repeatedly select the set that covers the most uncovered elements.

Greedy Set Cover Algorithm:

1. Initialize $\mathcal{C} = \emptyset$ and $R = U$.
2. While $R \neq \emptyset$:
   (a) Select $S_i \in \mathcal{S}$ that maximizes $|S_i \cap R|$.
   (b) Add S_i to $\mathcal{C}$.
   (c) Update $R \leftarrow R \setminus S_i$.
3. Return $\mathcal{C}$.

To analyze this algorithm, we use a charging argument that assigns costs to elements as they are covered. This is a powerful technique: instead of directly bounding the number of sets, we assign a cost to each element and show the total cost is bounded. The beauty of this approach is that it reduces a combinatorial counting problem to an accounting problem.

The analysis uses the following bound on the harmonic numbers $H_n = \sum_{i=1}^{n} \frac{1}{i}$. It corresponds to a special case of the Euler–Maclaurin summation formula. Its basic message is that oftentimes it is possible to exchange an integral for a sum.

Lemma 1.2. For all $n \in \mathbb{N}$, we have $\log(n+1) \leq H_n \leq 1 + \log n$.

Proof. For the upper bound, we calculate:

$$\begin{aligned} H_n &= \sum_{i=1}^{n} \frac{1}{i} = 1 + \sum_{i=2}^{n} \frac{1}{i} \int_{i-1}^{i} 1 \, dx = 1 + \sum_{i=2}^{n} \int_{i-1}^{i} \frac{1}{i} \, dx \leq 1 + \sum_{i=2}^{n} \int_{i-1}^{i} \frac{1}{x} \, dx \\ &= 1 + \int_{1}^{n} \frac{1}{x} \, dx = 1 + \log n \end{aligned}$$

For the lower bound, we note:

$$\begin{aligned} H_n &= \sum_{i=1}^{n} \frac{1}{i} = \sum_{i=1}^{n} \frac{1}{i} \int_{i}^{i+1} 1 \, dx = \sum_{i=1}^{n} \int_{i}^{i+1} \frac{1}{i} \, dx \geq \sum_{i=1}^{n} \int_{i}^{i+1} \frac{1}{x} \, dx \\ &= \int_{1}^{n+1} \frac{1}{x} \, dx = \log(n+1) \end{aligned}$$

This concludes the proof. □

Theorem 1.4. The greedy algorithm for set cover is an H_n-approximation, where $H_n = \sum_{i=1}^{n} \frac{1}{i}$ is the n-th harmonic number. Since $H_n \leq \ln n + 1$, the algorithm achieves an $O(\log n)$-approximation.

Proof. We assign a cost to each element when it is covered by the greedy algorithm. Consider the iteration when the greedy algorithm selects a set S covering t new elements. We charge a cost of $1/t$ to each of these newly covered elements, so the total cost assigned in this iteration is exactly 1. Since the algorithm selects exactly $|\mathcal{C}|$ sets, and each set is "charged" a total cost of 1, the total cost equals $|\mathcal{C}|$.

Let k denote the number of uncovered elements remaining before this iteration. Since an optimal solution must cover all n elements using at most OPT sets, by the pigeonhole principle at least one of the sets in the optimal solution must cover at least k/OPT of the remaining uncovered elements. The greedy algorithm selects the set covering the most uncovered elements, so it covers at least k/OPT elements, that is, $t \geq k/\text{OPT}$.

Therefore, the cost charged per element in this iteration is:

$$\frac{1}{t} \leq \frac{\text{OPT}}{k}$$

Now consider the perspective of a single element. Think about the element that is the j-th element to be covered, counting backwards from the end. That is, the last element covered has $j = 1$, the second-to-last has $j = 2$, and so on. When this element is covered, there are at least j elements still uncovered (including itself), so $k \geq j$. Thus, the cost assigned to this element is at most:

$$\frac{1}{t} \leq \frac{\text{OPT}}{k} \leq \frac{\text{OPT}}{j}$$

Summing over all n elements, the total cost assigned is:

$$\text{Cost} \leq \sum_{j=1}^{n} \frac{\text{OPT}}{j} = \text{OPT} \sum_{j=1}^{n} \frac{1}{j} = \text{OPT} \cdot H_n$$

Since each set selected by the greedy algorithm is assigned a total cost of 1 (distributed among the elements it covers), and the algorithm selects $|\mathcal{C}|$ sets, we have:

$$|\mathcal{C}| = \text{Cost} \leq \text{OPT} \cdot H_n$$

By Lemma 1.2, we have $H_n \leq 1 + \ln n$, completing the proof. □

The charging argument is a fundamental technique in approximation algorithms. We will see it again in other contexts. The key idea is always the same: assign costs to objects (here, elements), argue that greedy makes good local decisions about these costs, and sum up to get a global bound.

Just as with vertex cover, it is natural to ask whether we can improve upon the logarithmic approximation ratio for set cover. The answer is largely negative.

Feige [3] proved that set cover cannot be approximated within a factor of $(1 - \varepsilon) \ln n$ for any $\varepsilon > 0$, unless NP has slightly unusual complexity (specifically, unless NP $\subseteq$ DTIME($n^{O(\log \log n)}$)). This shows that the greedy algorithm's $O(\log n)$-approximation is essentially optimal.

This is a rare and beautiful case where we fully understand the approximability of a problem: the greedy algorithm achieves $\ln n$ approximation, and this is the best possible unless unusual complexity-theoretic collapses occur.

Two natural structural parameters control how far we can improve upon the $O(\log n)$ ratio for restricted

instances: the *size* of the sets and the *frequency* of the elements.

If every set has size at most K, the greedy analysis tightens immediately. In the proof of Theorem 1.4, when a set S covers t new elements, each is charged $1/t$. Since $t \leq K$, the maximum charge any element receives across all iterations is bounded by $H_K = \sum_{i=1}^{K} 1/i$ rather than H_n. So the greedy algorithm achieves an H_K-approximation, which is $O(\log K)$—an improvement when $K \ll n$, but still logarithmic.

A stronger improvement comes from bounding the frequency. Define the *frequency* of an element as the number of sets containing it. If every element has frequency at most f, we can obtain an f-approximation by a simple rounding argument. Consider any optimal solution $\mathcal{C}^*$ with $|\mathcal{C}^*| = \text{OPT}$. For each element $e \in U$, assign it a weight of $1/f$. Since e belongs to at most f sets, the total weight assigned to elements of any single set S_i is $|S_i|/f$. The total weight across all elements is n/f. On the other hand, since $\mathcal{C}^*$ covers all elements, summing the weights over sets in $\mathcal{C}^*$ gives at least n/f, so $\sum_{S \in \mathcal{C}^*} |S|/f \geq n/f$, which means $\sum_{S \in \mathcal{C}^*} |S| \geq n$. Now consider the algorithm that simply selects every set containing at least one uncovered element, greedily avoiding redundant sets. More directly: pick any uncovered element e; it belongs to at most f sets, and at least one of these sets is in $\mathcal{C}^*$. This means that any *maximal* sub-collection that covers U uses at most $f \cdot \text{OPT}$ sets, since each set in $\mathcal{C}^*$ "accounts for" at most f sets in our solution.

This explains why vertex cover admits a 2-approximation. Vertex cover is a special case of set cover (Lemma 1.1) where each element (edge) has frequency exactly 2: every edge belongs to exactly the two sets corresponding to its endpoints. The maximal matching algorithm from Theorem 1.3 is precisely an instantiation of the frequency-based argument with $f = 2$. Note also that the greedy maximum degree algorithm for vertex cover is the greedy set cover algorithm applied to this instance, so Theorem 1.4 gives an alternative $O(\log n)$ proof of Theorem 1.1.

The difference in approximability between vertex cover and general set cover thus comes down to structure. For vertex cover, we have a 2-approximation and suspect (via UGC) that this is optimal, but without UGC we only know a hardness factor of 1.3606. For set cover, we have matching upper and lower bounds of $\Theta(\log n)$, giving a complete picture of the problem's approximability.

1.4 Subset Sum

The vertex cover and set cover problems we've seen so far achieve constant-factor or logarithmic approximations. Can we do better? For some problems, the answer is yes: we can achieve arbitrarily good approximations.

The subset sum problem provides our first example of a PTAS and an FPTAS. Given positive integers $a_1, \ldots, a_n$ and a target T, we want to find a subset $S \subseteq \{1, \ldots, n\}$ such that $\sum_{i \in S} a_i$ is as close to T as possible without exceeding it.

The standard dynamic programming approach maintains a table $\text{DP}[i][s]$ which is true if we can achieve sum exactly s using a subset of the first i items. The recurrence is:

$$\text{DP}[i][s] = \text{DP}[i-1][s] \vee \text{DP}[i-1][s - a_i]$$

This runs in time $O(nT)$, which is pseudo-polynomial: it's polynomial in the value of T but exponential in the number of bits needed to represent T.

The key idea is to reduce the number of distinct sums we track by "trimming" the list of reachable sums.

For any $\delta > 0$, we say two values s and s' are δ-close if $s \leq s' \leq (1+\delta)s$. When maintaining our list of reachable sums, whenever we have two values that are δ-close, we keep only the smaller one.

Set $\delta = \varepsilon/(2n)$. We maintain a list L_i of reachable sums using items from $\{a_1, \ldots, a_i\}$, but we keep L_i trimmed so that no two values are δ-close.

Algorithm:

1. Initialize $L_0 = \{0\}$.
2. For $i = 1$ to n:

(a) Compute $L_i' = L_{i-1} \cup \{s + a_i : s \in L_{i-1}\}$.

(b) Trim L_i' to obtain L_i:

- Sort L_i' in increasing order.
- Initialize $L_i = \emptyset$ and last $= 0$.
- For each $s \in L_i'$ in sorted order:
  - If $s > (1 + \delta) \cdot$ last, add s to L_i and set last $= s$.

3. Return the largest value in L_n that does not exceed T.

Theorem 1.5. For any $\varepsilon > 0$, the above algorithm is a $(1 + \varepsilon)$-approximation for subset sum, running in time $O(n^3/\varepsilon)$.

Proof. We first bound the size of each list. We claim that $|L_i| \leq \frac{2n}{\varepsilon} \log T$ for all i. The values in L_i are sorted and consecutive values differ by a factor of at least $(1 + \delta)$. If $L_i = \{s_1, s_2, \ldots, s_k\}$ with $s_1 < s_2 < \cdots < s_k$, then:

$$s_j \geq s_1 \cdot (1 + \delta)^{j-1} \geq (1 + \delta)^{j-1}$$

Since $s_k \leq \sum_{i=1}^{n} a_i \leq nT$, we have:

$$(1 + \delta)^{k-1} \leq nT$$

Taking logarithms:

$$(k - 1) \log(1 + \delta) \leq \log(nT)$$

Using $\log(1 + \delta) \geq \delta/2$ for small δ:

$$k \leq 1 + \frac{\log(nT)}{\delta/2} = 1 + \frac{2\log(nT)}{\delta} = 1 + \frac{2\log(nT)}{\varepsilon/(2n)} = 1 + \frac{4n\log(nT)}{\varepsilon}$$

Therefore $|L_i| = O(n \log T/\varepsilon)$.

We now bound the approximation error. Let OPT_i denote the maximum sum achievable using items from $\{a_1, \ldots, a_i\}$ that does not exceed T. We prove by induction that L_i contains a value y_i such that:

$$y_i \geq \frac{\mathrm{OPT}_i}{(1 + \delta)^i}$$

Base case: $L_0 = \{0\}$ and $\mathrm{OPT}_0 = 0$, so the claim holds.

Inductive step: Assume the claim holds for $i - 1$. By the inductive hypothesis, there exists $y_{i-1} \in L_{i-1}$ with $y_{i-1} \geq \mathrm{OPT}_{i-1}/(1 + \delta)^{i-1}$.

If $\mathrm{OPT}_i = \mathrm{OPT}_{i-1}$ (item a_i is not used in the optimal solution), then y_{i-1} is added to L_i'. During trimming, either y_{i-1} itself survives in L_i, or it is replaced by a value $z \leq y_{i-1}$ with $z \geq y_{i-1}/(1 + \delta)$. In either case, there exists $y_i \in L_i$ with $y_i \geq y_{i-1}/(1 + \delta) \geq \mathrm{OPT}_i/(1 + \delta)^i$.

Otherwise, $\mathrm{OPT}_i = \mathrm{OPT}_{i-1} + a_i$ for some optimal subset that includes a_i. Let OPT'_{i-1} be the sum of the optimal subset of $\{a_1, \ldots, a_{i-1}\}$ that is used in this optimal solution for OPT_i. Then $\mathrm{OPT}_i = \mathrm{OPT}'_{i-1} + a_i$.

By the inductive hypothesis, there exists $y'_{i-1} \in L_{i-1}$ satisfying $y'_{i-1} \geq \mathrm{OPT}'_{i-1}/(1 + \delta)^{i-1}$. The value $y'_{i-1} + a_i$ is computed and added to L_i'.

During trimming, either $y'_{i-1} + a_i$ itself survives, or it is removed because there is a smaller value z within a factor $(1+\delta)$ of it. In either case, there exists $y_i \in L_i$ with:

$$y_i \geq \frac{y'_{i-1} + a_i}{1+\delta} \geq \frac{\mathrm{OPT}'_{i-1}/(1+\delta)^{i-1} + a_i}{1+\delta} \geq \frac{\mathrm{OPT}_i}{(1+\delta)^i}$$

Therefore, the largest value in L_n is at least:

$$\frac{\mathrm{OPT}}{(1+\delta)^n} = \frac{\mathrm{OPT}}{(1+\varepsilon/(2n))^n}$$

Using $(1+x)^n \leq e^{nx}$ for $x > 0$:

$$(1+\varepsilon/(2n))^n \leq e^{\varepsilon/2}$$

For $\varepsilon \leq 1$, we have $\varepsilon/2 \leq \ln(1+\varepsilon)$ (since $\ln(1+x) \geq x - x^2/2 \geq x/2$ for $x \in [0,1]$), so $e^{\varepsilon/2} \leq 1+\varepsilon$. Therefore, the algorithm returns a value at least $\mathrm{OPT}/(1+\varepsilon)$. Since subset sum is a maximization problem, this means $\mathrm{ALG} \geq \mathrm{OPT}/(1+\varepsilon)$, so the algorithm is a $(1+\varepsilon)$-approximation.

For the running time, each iteration i processes the list L_{i-1}, whose size is $O(n \log T/\varepsilon)$. Computing L'_i takes $O(|L_{i-1}|)$ time, trimming takes $O(|L'_i|)$ time, and sorting takes $O(|L'_i| \log |L'_i|) = O((n \log T/\varepsilon) \log n)$ time.

Over n iterations, the total time is:

$$O\left(n \cdot \frac{n \log T}{\varepsilon} \log n\right) = O\left(\frac{n^2 \log n \log T}{\varepsilon}\right)$$

If we assume $T \leq 2^{poly(n)}$ (polynomial number of bits), then $\log T = O(n^c)$ for some constant c, giving total time $O(n^3/\varepsilon)$. □

This example illustrates that some NP-hard problems admit FPTAS: for any $\varepsilon > 0$, we achieve approximation ratio $1+\varepsilon$ in time polynomial in both n and $1/\varepsilon$. The key technique is controlled loss of precision: by trimming values that are very close to each other, we reduce the state space while losing only a small multiplicative factor in solution quality.

September 7, 2026

Lecture 2: Approximation Algorithms II

In the previous lecture, we studied approximation algorithms for vertex cover, set cover, and subset sum. We saw constant-factor approximations, logarithmic approximations, and an FPTAS. In this lecture, we continue with three more problems. The metric traveling salesman problem illustrates how structural assumptions (the triangle inequality) can turn an inapproximable problem into one admitting a constant-factor guarantee. The 0/1 knapsack problem gives a second example of an FPTAS, this time via profit scaling rather than trimming. Load balancing on identical machines shows how a simple algorithmic refinement (sorting the input) can dramatically improve an approximation ratio.

2.1 Metric Traveling Salesman Problem

The traveling salesman problem (TSP) asks for a minimum-cost tour that visits every vertex in a graph exactly once and returns to the start. Given a complete graph K_n on n vertices with edge costs $c(u, v) \geq 0$, we seek a Hamiltonian cycle of minimum total cost.

In the general case, TSP is extremely hard to approximate. Sahni and Gonzalez [7] showed that no finite approximation ratio is achievable.

Theorem 2.1. Unless P = NP, for any polynomial-time computable function $\rho(n)$, there is no polynomial-time $\rho(n)$-approximation for the general TSP.

Proof. We reduce from the Hamiltonian Cycle problem. Given a graph $G = (V, E)$ on n vertices, construct a complete graph K_n on the same vertex set with costs:

$$c(u, v) = \begin{cases} 1 & \text{if } \{u, v\} \in E, \\ n \cdot \rho(n) + 1 & \text{otherwise.} \end{cases}$$

If G has a Hamiltonian cycle, the optimal TSP tour has cost n. If G has no Hamiltonian cycle, any tour must use at least one non-edge, so the optimal tour has cost at least $n - 1 + n\rho(n) + 1 = n \cdot \rho(n) + n > \rho(n) \cdot n$.

A $\rho(n)$-approximation algorithm would produce a tour of cost at most $\rho(n) \cdot n$ when OPT $= n$, distinguishing the two cases. Since Hamiltonian Cycle is NP-complete, no such algorithm exists unless P = NP. □

Why is the general case so hopeless? The reduction above constructs instances where non-edges receive artificially enormous costs, creating a gap between "yes" and "no" instances that no algorithm can bridge. The triangle inequality prevents exactly this kind of construction: if there is a path of cost k between u and w, then the direct edge $c(u, w)$ can cost at most k. Formally, the metric TSP requires edge costs to satisfy:

$$c(u, w) \leq c(u, v) + c(v, w) \quad \text{for all } u, v, w \in V.$$

This restriction is natural in many applications (e.g., Euclidean distances). For approximation, it has a crucial algorithmic consequence: *shortcuts are free*. If a walk visits vertices $u, v_1, \ldots, v_k, w$, we can skip the intermediate vertices and go directly from u to w without increasing the cost, since $c(u, w) \leq c(u, v_1) + c(v_1, v_2) + \cdots + c(v_k, w)$. This means that any closed walk through all vertices can be converted into a Hamiltonian cycle of no greater cost.

So the problem reduces to: find a cheap closed walk that visits every vertex. This is easier than finding a Hamiltonian cycle directly, because walks are allowed to revisit vertices.

As in the vertex cover analysis from the previous lecture, we start by looking for a good lower bound on OPT. An optimal tour visits all n vertices and returns to the start. Removing one edge from this tour yields a spanning path, which is in particular a spanning tree. So every tour contains a spanning tree, and the minimum spanning tree gives a lower bound. Crucially, MSTs can be computed efficiently: sort the edges by cost and greedily add each edge that does not create a cycle (Kruskal's algorithm), which runs in $O(m \log n)$ time.

Lemma 2.1. Let T be a minimum spanning tree of K_n. Then $c(T) \leq \mathrm{OPT}$.

Proof. Removing any edge from an optimal tour yields a spanning tree of cost at most OPT. Since T is a minimum spanning tree, $c(T)$ is at most the cost of any spanning tree, so $c(T) \leq \mathrm{OPT}$. □

Now the strategy is clear: use the MST as a backbone and turn it into a closed walk (which we can then shortcut into a tour). A tree is not a closed walk, but we can make it one by traversing each edge twice, once in each direction. This gives a multigraph where every vertex has even degree, which admits an Eulerian tour. Such a tour can be found in linear time: start at any vertex and greedily follow unused edges until returning to the start; whenever the resulting circuit misses some edges, splice in a sub-circuit from a vertex that still has unused edges, and repeat until all edges are used.

Algorithm (MST-doubling):

1. Compute a minimum spanning tree T of K_n.
2. Double every edge of T to obtain a multigraph H where every vertex has even degree.
3. Find an Eulerian tour of H.
4. Shortcut the Eulerian tour: skip any vertex already visited, going directly to the next unvisited vertex.

Theorem 2.2. The MST-doubling algorithm is a 2-approximation for metric TSP.

Proof. The Eulerian tour of H has cost $c(H) = 2 \cdot c(T) \leq 2 \cdot \mathrm{OPT}$ by Lemma 2.1. Shortcutting can only decrease the cost by the triangle inequality. Therefore, the final tour has cost at most $2 \cdot \mathrm{OPT}$. □

The factor of 2 comes from doubling *every* edge of the MST. But why did we double edges in the first place? Only to make all degrees even, so that an Eulerian tour exists. If a vertex already has even degree, doubling its incident edges is wasteful. The vertices that actually need fixing are precisely those with *odd* degree. The key idea, due to Christofides [6] and independently Serdyukov [27], is to add edges only where needed: a minimum-weight perfect matching on the odd-degree vertices.

Algorithm (Christofides–Serdyukov):

1. Compute a minimum spanning tree T of K_n.
2. Let S be the set of vertices with odd degree in T.
3. Compute a minimum-weight perfect matching M on the complete graph induced by S.
4. Combine T and M to form a multigraph $H = T \cup M$.
5. Find an Eulerian tour of H and shortcut repeated vertices.

Why does a perfect matching on S exist? The sum of degrees in any graph is even, so the number of odd-degree vertices $|S|$ is even. Since we compute the matching on the *complete* graph induced by S, any pairing of the $|S|$ vertices into $|S|/2$ pairs gives a perfect matching. Finding a minimum-weight such matching is a nontrivial algorithmic problem: the main difficulty is that, unlike in bipartite graphs, augmenting paths in general graphs can traverse odd cycles, which Edmonds [30] resolved by "shrinking" these cycles (called *blossoms*) into single vertices. The resulting algorithm runs in $O(n^3)$ time. Adding the matching edges to T increases the degree of each vertex in S by exactly one, making all degrees even and enabling the Eulerian tour.

Lemma 2.2. The minimum-weight perfect matching M on S satisfies $c(M) \leq \mathrm{OPT}/2$.

Proof. Consider an optimal TSP tour and restrict it to the vertices in S. Since the tour visits every vertex, we can shortcut it to obtain a tour on S alone. By the triangle inequality, this restricted tour has cost at most OPT.

Let $|S| = 2k$. The restricted tour on S visits $2k$ vertices in some order $s_1, s_2, \ldots, s_{2k}, s_1$. Partition the edges of this tour into two perfect matchings:

$$M_1 = \{\{s_1, s_2\}, \{s_3, s_4\}, \ldots, \{s_{2k-1}, s_{2k}\}\}$$

$$M_2 = \{\{s_2, s_3\}, \{s_4, s_5\}, \ldots, \{s_{2k}, s_1\}\}$$

Both M_1 and M_2 are perfect matchings on S, and their total cost satisfies $c(M_1) + c(M_2) \leq \mathrm{OPT}$. Therefore, $\min(c(M_1), c(M_2)) \leq \mathrm{OPT}/2$. Since M is a minimum-weight perfect matching, $c(M) \leq \mathrm{OPT}/2$. □

Theorem 2.3. The Christofides–Serdyukov algorithm is a $\frac{3}{2}$-approximation for metric TSP.

Proof. The multigraph $H = T \cup M$ has cost $c(H) = c(T) + c(M)$. By Lemma 2.1, $c(T) \leq \mathrm{OPT}$. By Lemma 2.2, $c(M) \leq \mathrm{OPT}/2$. Therefore:

$$c(H) = c(T) + c(M) \leq \mathrm{OPT} + \frac{\mathrm{OPT}}{2} = \frac{3}{2} \cdot \mathrm{OPT}$$

As in the MST-doubling analysis, shortcutting the Eulerian tour of H can only decrease the cost by the triangle inequality. The final tour has cost at most $\frac{3}{2} \cdot \mathrm{OPT}$. □

The $\frac{3}{2}$-approximation ratio stood as the best known for nearly fifty years. In 2020, Karlin, Klein, and Oveis Gharan [28] achieved a breakthrough by obtaining a $(\frac{3}{2} - \varepsilon_0)$-approximation for some small constant $\varepsilon_0 > 0$, the first improvement over $\frac{3}{2}$. On the hardness side, metric TSP is known to be APX-hard: there is no PTAS unless P = NP.

2.2 Knapsack

The 0/1 knapsack problem is one of the most fundamental problems in combinatorial optimization. We are given n items, where item i has weight $w_i > 0$ and profit $p_i > 0$, along with a knapsack capacity W. The goal is to find a subset $S \subseteq \{1, \ldots, n\}$ maximizing $\sum_{i \in S} p_i$ subject to $\sum_{i \in S} w_i \leq W$. We may assume without loss of generality that $w_i \leq W$ for all i, since items that do not fit individually can be discarded.

Subset sum is the special case where $w_i = p_i$ for all i and W is the target value. In the previous lecture,

we gave an FPTAS for subset sum using a trimming technique. Here we present a different approach to an FPTAS for the more general knapsack problem, based on profit scaling. The idea is remarkably simple: we already know how to solve knapsack *exactly* by dynamic programming, but the running time depends on the size of the profit values. If we could make these values smaller, the DP would be faster. Rounding profits down loses some precision, but if we control the rounding error, the solution remains close to optimal.

The standard DP for knapsack indexes by weight and runs in time $O(nW)$. For our purposes, we index by *profit* instead. Let $A[i][p]$ be the minimum weight to achieve profit exactly p using items from $\{1, \ldots, i\}$. The recurrence is:

$$A[i][p] = \min\big(A[i-1][p],\ A[i-1][p-p_i] + w_i\big)$$

with base cases $A[0][0] = 0$ and $A[0][p] = \infty$ for $p > 0$.

The optimal profit is the largest p such that $A[n][p] \le W$. Let $P = \sum_{i=1}^{n} p_i$ denote the total profit. The table has $O(nP)$ entries, so this DP runs in $O(nP)$ time, which is pseudo-polynomial: polynomial in the *values* of the profits but potentially exponential in the input size.

Why index by profit rather than weight? Because we are about to *scale down* the profits, making P polynomial in n and $1/\varepsilon$. We have no such control over W.

How much can we afford to round? Replacing each profit p_i by $\lfloor p_i/K \rfloor$ for some scaling factor K introduces a per-item error of at most K. Over at most n items, the total error is at most nK. For this to be at most $\varepsilon \cdot \mathrm{OPT}$, we need $nK \le \varepsilon \cdot \mathrm{OPT}$. We don't know OPT, but we do know that $\mathrm{OPT} \ge p_{\max}$ (since every item fits individually). Setting $K = \varepsilon \cdot p_{\max}/n$ guarantees $nK = \varepsilon \cdot p_{\max} \le \varepsilon \cdot \mathrm{OPT}$.

Algorithm (FPTAS for Knapsack):

1. Let $p_{\max} = \max_i p_i$.
2. Set the scaling factor $K = \frac{\varepsilon \cdot p_{\max}}{n}$.
3. For each item i, define the scaled profit $p'_i = \left\lfloor \frac{p_i}{K} \right\rfloor$.
4. Run the exact DP on the scaled profits $p'_1, \ldots, p'_n$ with the original weights $w_1, \ldots, w_n$ and capacity W.
5. Return the subset found by the DP.

Theorem 2.4. For any $\varepsilon > 0$, the above algorithm is a $(1+\varepsilon)$-approximation for knapsack, running in time $O(n^3/\varepsilon)$.

Proof. Each scaled profit satisfies:

$$p'_i = \left\lfloor \frac{p_i}{K} \right\rfloor \le \frac{p_i}{K} \le \frac{p_{\max}}{K} = \frac{n}{\varepsilon}$$

The total scaled profit is $P' = \sum_{i=1}^{n} p'_i \le n^2/\varepsilon$. The DP runs in $O(nP') = O(n^3/\varepsilon)$ time, which is polynomial in n and $1/\varepsilon$.

For the approximation guarantee, let S^* be an optimal solution with profit $\mathrm{OPT} = \sum_{i \in S^*} p_i$. Since S^* satisfies the weight constraint, the DP on scaled profits finds a solution S with $\sum_{i \in S} p'_i \ge \sum_{i \in S^*} p'_i$.

For each item i, the floor satisfies $p'_i \ge p_i/K - 1$, so $p'_i \cdot K \ge p_i - K$. Therefore the true profit of S satisfies:

$$\sum_{i \in S} p_i \ge \sum_{i \in S} p'_i \cdot K \ge \sum_{i \in S^*} p'_i \cdot K \ge \sum_{i \in S^*} (p_i - K) = \mathrm{OPT} - |S^*| \cdot K$$

Since $|S^*| \le n$ and $K = \varepsilon \cdot p_{\max}/n$, the total rounding error is $|S^*| \cdot K \le \varepsilon \cdot p_{\max} \le \varepsilon \cdot \mathrm{OPT}$. Hence $\mathrm{ALG} \ge (1-\varepsilon) \cdot \mathrm{OPT}$. To obtain a $(1+\varepsilon)$-approximation, we run the algorithm with parameter $\varepsilon/2$

instead of ε. This gives ALG $\geq (1-\varepsilon/2)\cdot\text{OPT} \geq \text{OPT}/(1+\varepsilon)$, since $(1-\varepsilon/2)(1+\varepsilon) = 1+\varepsilon/2-\varepsilon^2/2 \geq 1$ for $\varepsilon \leq 1$. The running time becomes $O(n^3/(\varepsilon/2)) = O(n^3/\varepsilon)$. □

The profit-scaling technique, introduced by Ibarra and Kim [8], is conceptually simpler than the trimming approach we saw for subset sum. Both achieve an FPTAS, but scaling has the advantage of reducing to an existing exact algorithm as a black box: we simply change the input and run the same DP.

2.3 Load Balancing

We now turn to a scheduling problem, which is NP-hard (by reduction from the Partition problem). We have m identical machines and n jobs with processing times $t_1, t_2, \ldots, t_n$. Each job must be assigned to exactly one machine, and the load of a machine is the total processing time of jobs assigned to it. The objective is to minimize the *makespan*: the maximum load over all machines.

Formally, given an assignment $\sigma\colon \{1,\ldots,n\} \to \{1,\ldots,m\}$, the makespan is:

$$\max_{j=1}^{m} \sum_{i:\sigma(i)=j} t_i$$

As before, we start by identifying lower bounds on the optimal makespan. Two are immediate.

Lemma 2.3. The optimal makespan satisfies:

$$\text{OPT} \geq \frac{1}{m}\sum_{i=1}^{n} t_i \quad \text{and} \quad \text{OPT} \geq \max_{i=1}^{n} t_i$$

Proof. The total work $\sum_i t_i$ is distributed among m machines, so by averaging the most loaded machine has load at least $\frac{1}{m}\sum_i t_i$. The second bound holds because whichever machine processes the longest job has load at least $\max_i t_i$. □

Neither bound alone is tight in general, but together they capture the two fundamentally different sources of difficulty: too much total work, or a single job that is too large.

The simplest scheduling heuristic is list scheduling, introduced by Graham [9]: process jobs one by one, always assigning the next job to the least loaded machine. This is the greedy algorithm that tries to keep loads balanced at every step.

Algorithm (List Scheduling):

1. Process jobs in the order $1, 2, \ldots, n$.
2. Assign each job to the machine with the current smallest load.

To analyze list scheduling, we focus on the bottleneck machine and the moment the last job lands on it. The makespan decomposes as the load L before that job plus the job's processing time t_ℓ. We can bound each term separately using our two lower bounds.

Theorem 2.5. List scheduling is a $(2 - 1/m)$-approximation for makespan minimization.

Proof. Let M_j be the machine with the maximum load at the end, let t_ℓ be the last job assigned to it, and let L be the load of M_j just before job ℓ was assigned. The makespan is $L + t_\ell$.

When job ℓ was assigned, M_j had the smallest load among all machines, so every machine had

load at least L. The total load on all m machines at that point is $\sum_{i<\ell} t_i \le \sum_i t_i - t_\ell$, and since M_j has the smallest load, $m \cdot L \le \sum_i t_i - t_\ell$. Therefore:

$$L \le \frac{1}{m}\left(\sum_{i=1}^{n} t_i - t_\ell\right) \le \mathrm{OPT} - \frac{t_\ell}{m}$$

by Lemma 2.3. Also, $t_\ell \le \max_i t_i \le \mathrm{OPT}$ by the same lemma. Therefore:

$$\begin{aligned}\mathrm{ALG} = L + t_\ell &\le \mathrm{OPT} - \frac{t_\ell}{m} + t_\ell = \mathrm{OPT} + \left(1 - \frac{1}{m}\right) t_\ell \\ &\le \mathrm{OPT} + \left(1 - \frac{1}{m}\right) \mathrm{OPT} = \left(2 - \frac{1}{m}\right) \mathrm{OPT}\end{aligned}$$

□

The bound $(2 - 1/m)$ is tight for list scheduling. The worst case arises when a large job arrives last and gets piled onto an already loaded machine. Consider $m(m-1)$ jobs of size 1 followed by one job of size m. List scheduling distributes the small jobs evenly, giving each machine load $m - 1$, then assigns the large job to one of them, resulting in makespan $2m - 1$. The optimal schedule places the large job alone on one machine and distributes the small jobs among the remaining $m - 1$ machines, achieving makespan m. The ratio is $(2m-1)/m = 2 - 1/m$.

The problem is clear: list scheduling is oblivious to job sizes, so a large job can arrive last and ruin the balance. The fix is equally clear: process large jobs first, when the machines are still lightly loaded. Graham [10] observed that this simple modification leads to a significantly improved guarantee.

Algorithm (Longest Processing Time first, LPT):

1. Sort jobs so that $t_1 \ge t_2 \ge \cdots \ge t_n$.
2. Apply list scheduling in this order.

The key consequence of sorting is that the last job assigned to the bottleneck machine is now the *smallest* job involved, not an arbitrarily large one. This lets us replace the bound $t_\ell \le \mathrm{OPT}$ with the much tighter $t_\ell \le \mathrm{OPT}/3$.

Theorem 2.6. LPT is a $(4/3 - 1/(3m))$-approximation for makespan minimization.

Proof. If $n \le m$, each job gets its own machine and the makespan equals $\max_i t_i = \mathrm{OPT}$. So assume $n > m$.

Let M_j be the bottleneck machine and let t_ℓ be the last job assigned to it. If M_j received only one job, then $\mathrm{ALG} = t_\ell \le \mathrm{OPT}$ and we are done. Otherwise, M_j received at least two jobs, so $\ell \ge m + 1$ (the first m jobs are assigned to distinct machines), and therefore $t_\ell \le t_{m+1}$.

We distinguish two cases.

Case 1: $t_\ell \le \mathrm{OPT}/3$. Using the same bound on L as in Theorem 2.5:

$$\begin{aligned}\mathrm{ALG} = L + t_\ell &\le \mathrm{OPT} - \frac{t_\ell}{m} + t_\ell = \mathrm{OPT} + \left(1 - \frac{1}{m}\right) t_\ell \\ &\le \mathrm{OPT} + \left(1 - \frac{1}{m}\right) \frac{\mathrm{OPT}}{3} = \left(\frac{4}{3} - \frac{1}{3m}\right) \mathrm{OPT}\end{aligned}$$

Case 2: $t_\ell > \mathrm{OPT}/3$. Since jobs are sorted in decreasing order and $\ell \ge m + 1$, we have

$t_i \geq t_{m+1} \geq t_\ell > \text{OPT}/3$ for all $i \in \{1, \ldots, m+1\}$. In any schedule, no machine can hold three or more of these $m+1$ large jobs (their total processing time would exceed OPT). By pigeonhole ($m+1$ large jobs, m machines, at most 2 per machine), exactly one machine holds two large jobs and the remaining $m-1$ machines each hold one.

We claim that LPT is optimal. Consider any optimal schedule. The makespan is at least the load of the machine holding two large jobs, say $t_a + t_b$ with $a < b$ (so $t_a \geq t_b$). Now suppose in some other schedule, t_1 is paired with t_k where $k \leq m$. Then $t_k \geq t_{m+1}$, so $t_1 + t_k \geq t_1 + t_{m+1}$: pairing t_1 with any other job does not decrease the load of the heavy machine. The optimal pairing is therefore to place $t_1, \ldots, t_m$ on separate machines and pair t_{m+1} with the lightest, which is t_m. This is exactly what LPT does: it assigns the first m jobs round-robin, then t_{m+1} joins the least loaded machine (holding t_m). The remaining small jobs $t_{m+2}, \ldots, t_n$ are each at most $t_{m+1} \leq \text{OPT}$, and LPT assigns each to the current least loaded machine, which cannot exceed the optimal makespan. Therefore $\text{ALG} = \text{OPT}$.

In both cases, $\text{ALG} \leq (4/3 - 1/(3m)) \cdot \text{OPT}$. □

The 4/3 ratio of LPT is a significant improvement over the factor of 2 from plain list scheduling, obtained simply by sorting the input. For the load balancing problem, Hochbaum and Shmoys [29] showed that a PTAS exists: for any $\varepsilon > 0$, a $(1+\varepsilon)$-approximation can be found in polynomial time, though the algorithm is considerably more involved than the simple LPT rule.

September 14, 2026

Lecture 3: Approximation Algorithms III

In the previous two lectures, we designed approximation algorithms using combinatorial techniques: greedy algorithms, matchings, dynamic programming, and scaling. In this lecture, we add two more tools to our repertoire. First, we introduce linear programming (LP) relaxation: formulate the problem as an integer program, relax the integrality constraints, solve the resulting LP, and round the fractional solution. We illustrate this on weighted vertex cover and weighted set cover. Then, we study MAX-CUT via semidefinite programming (SDP), a powerful generalization of LP relaxation.

3.1 Linear Programming Relaxation

A linear program (LP) asks to optimize a linear objective function subject to linear constraints. In its standard form, an LP minimizes (or maximizes) ${}^t c\, x$ subject to $Ax \leq b$ and $x \geq 0$, where $x \in \mathbb{R}^n$. Linear programs can be solved in polynomial time, for example by interior point methods.

An integer linear program (ILP) is an LP with the additional constraint that the variables must take integer values: $x \in \mathbb{Z}^n$. In general, solving an ILP is NP-hard. For combinatorial optimization problems, the variables are usually binary ($x \in \{0,1\}^n$), and we focus on this case.

The key idea behind LP relaxation is to drop the integrality constraints, obtaining a problem that is easier to solve and whose optimal value provides a bound on the true optimum.

Definition 3.1: LP relaxation

Given an ILP with binary variables $x_i \in \{0,1\}$, its LP relaxation is obtained by replacing each integrality constraint with the continuous constraint $0 \leq x_i \leq 1$.

For a minimization problem, the LP relaxation has a larger feasible region (every integer solution is also a fractional solution), so $\mathrm{OPT}_{\mathrm{LP}} \leq \mathrm{OPT}$. For a maximization problem, $\mathrm{OPT}_{\mathrm{LP}} \geq \mathrm{OPT}$. In both cases, the LP optimum provides a bound on the true optimum that we can exploit algorithmically.

Definition 3.2: Integrality gap

The integrality gap of an LP relaxation is the worst-case ratio between the optimal integer solution and the optimal fractional solution, taken over all instances. For a minimization problem, it is $\sup_I \frac{\mathrm{OPT}(I)}{\mathrm{OPT}_{\mathrm{LP}}(I)}$; for a maximization problem, it is $\inf_I \frac{\mathrm{OPT}(I)}{\mathrm{OPT}_{\mathrm{LP}}(I)}$.

The integrality gap characterizes the power of a relaxation: no rounding scheme can achieve an approximation ratio better than the integrality gap, and conversely, if a rounding scheme achieves cost at most $\alpha \cdot \mathrm{OPT}_{\mathrm{LP}}$ on every instance (for a minimization problem), then it is an α-approximation algorithm, since $\mathrm{OPT}_{\mathrm{LP}} \leq \mathrm{OPT}$. The integrality gap is therefore exactly the best approximation ratio achievable by rounding a given relaxation.

Not all LP relaxations are useful. For example, the natural LP relaxation for maximum-weight independent set (maximize $\sum_v w_v x_v$ subject to $x_u + x_v \leq 1$ for all edges, $0 \leq x_v \leq 1$) has an integrality gap of $n/2$ on the complete graph K_n: the LP sets every $x_v = 1/2$ for a value of $n/2$, while the integer optimum is 1. No rounding scheme can overcome such a gap. The problems below have much better integrality gaps.

3.2 Weighted Vertex Cover

We now revisit the vertex cover problem, this time in its weighted version. Given an undirected graph $G = (V, E)$ and vertex weights $w_v \geq 0$ for each $v \in V$, the goal is to find a vertex cover $C \subseteq V$ (a set such that

every edge has at least one endpoint in C) of minimum total weight $w(C) = \sum_{v \in C} w_v$.

In Lecture 1, the edge-picking algorithm (Theorem 1.3) gave a 2-approximation for unweighted vertex cover. However, that combinatorial approach does not directly extend to the weighted case, since picking both endpoints of an edge may include a very heavy vertex unnecessarily. LP relaxation provides a clean way to handle weights.

We formulate the problem as an ILP. For each vertex $v \in V$, introduce a variable $x_v \in \{0, 1\}$ indicating whether v is in the cover. The ILP is:

$$\begin{aligned} \text{minimize} \quad & \sum_{v \in V} w_v \, x_v \\ \text{subject to} \quad & x_u + x_v \geq 1 && \text{for all } \{u, v\} \in E \\ & x_v \in \{0, 1\} && \text{for all } v \in V \end{aligned}$$

The LP relaxation replaces $x_v \in \{0, 1\}$ with $0 \leq x_v \leq 1$:

$$\begin{aligned} \text{minimize} \quad & \sum_{v \in V} w_v \, x_v \\ \text{subject to} \quad & x_u + x_v \geq 1 && \text{for all } \{u, v\} \in E \\ & 0 \leq x_v \leq 1 && \text{for all } v \in V \end{aligned} \tag{1}$$

Let x^* be an optimal solution to the LP relaxation. We round this fractional solution to an integer solution by a simple threshold rule.

Algorithm:

1. Solve the LP relaxation (1) to obtain an optimal solution x^*.
2. For each vertex $v \in V$: include v in C if and only if $x_v^* \geq 1/2$.
3. Return C.

Theorem 3.1. The LP rounding algorithm is a 2-approximation for weighted vertex cover.

Proof. We must verify two things: that C is a valid vertex cover, and that its weight is at most $2 \cdot \mathrm{OPT}$.

For feasibility, consider any edge $\{u, v\} \in E$. The LP constraint gives $x_u^* + x_v^* \geq 1$. Therefore, at least one of x_u^* or x_v^* is at least $1/2$, so at least one of u or v is included in C. Hence C is a vertex cover.

For the cost bound, for every vertex $v \in C$, we have $x_v^* \geq 1/2$, so $1 \leq 2x_v^*$. Therefore:

$$w(C) = \sum_{v \in C} w_v \leq \sum_{v \in C} w_v \cdot 2x_v^* \leq 2 \sum_{v \in V} w_v \, x_v^* = 2 \cdot \mathrm{OPT}_{\mathrm{LP}} \leq 2 \cdot \mathrm{OPT}$$

This completes the proof. □

We now show that the factor of 2 is tight for this LP relaxation, meaning no rounding scheme can do better.

Theorem 3.2. The integrality gap of the vertex cover LP relaxation (1) is exactly 2.

Proof. Theorem 3.1 shows that the integrality gap is at most 2. For the lower bound, consider the complete graph K_n with unit weights $w_v = 1$ for all v. The fractional solution $x_v^* = 1/2$ for all v is feasible: for every edge $\{u, v\}$, we have $x_u^* + x_v^* = 1 \geq 1$. Its cost is $\mathrm{OPT}_{\mathrm{LP}} \leq n/2$. On the other hand, any integer vertex cover must include at least $n - 1$ vertices (omitting at most one vertex), so $\mathrm{OPT} = n - 1$. The ratio $\mathrm{OPT}/\mathrm{OPT}_{\mathrm{LP}} \geq (n-1)/(n/2) = 2(n-1)/n$, which approaches 2 as $n \to \infty$. □

We remark that the vertex cover LP has a remarkable structural property: every basic feasible solution of the LP relaxation (1) is half-integral, meaning that every variable takes a value in $\{0, 1/2, 1\}$. This is a consequence of the total dual integrality of the constraint matrix and explains why the simple threshold $1/2$ works so well.

3.3 Weighted Set Cover

In the weighted set cover problem, we are given a universe $U = \{e_1, \ldots, e_n\}$ of n elements, a collection $\mathcal{S} = \{S_1, \ldots, S_m\}$ of subsets of U, and a cost $c_j \geq 0$ for each set S_j. The goal is to find a subcollection of minimum total cost that covers every element, i.e., whose union is U. In Lecture 1, we showed that the greedy algorithm is an H_n-approximation (Theorem 1.4), and that this is essentially optimal [3]. LP relaxation gives a different, incomparable guarantee.

The *frequency* of an element e is the number of sets containing it: $f_e = |\{j : e \in S_j\}|$. Let $f = \max_e f_e$ be the maximum frequency.

We formulate the problem as an ILP with a variable $x_j \in \{0, 1\}$ for each set S_j:

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^{m} c_j \, x_j & & \\ \text{subject to} \quad & \sum_{j : e \in S_j} x_j \geq 1 & & \text{for all } e \in U \\ & x_j \in \{0, 1\} & & \text{for all } j \end{aligned} \tag{2}$$

The LP relaxation replaces $x_j \in \{0, 1\}$ with $0 \leq x_j \leq 1$. Let x^* be an optimal LP solution with cost $\mathrm{OPT}_{\mathrm{LP}} \leq \mathrm{OPT}$.

Algorithm:

1. Solve the LP relaxation to obtain x^*.
2. Include S_j in the cover if and only if $x_j^* \geq 1/f$.
3. Return the selected sets.

Theorem 3.3. The LP rounding algorithm is an f-approximation for set cover.

Proof. For feasibility, consider any element $e \in U$. The LP constraint gives $\sum_{j : e \in S_j} x_j^* \geq 1$. This sum has at most $f_e \leq f$ terms, so at least one of them satisfies $x_j^* \geq 1/f$. That set is included, and e is covered.

For the cost bound, every selected set has $x_j^* \geq 1/f$, so $1 \leq f \cdot x_j^*$. Therefore:

$$\sum_{j \in \text{cover}} c_j \leq \sum_{j \in \text{cover}} c_j \cdot f \cdot x_j^* \leq f \sum_{j=1}^{m} c_j \, x_j^* = f \cdot \mathrm{OPT}_{\mathrm{LP}} \leq f \cdot \mathrm{OPT}$$

□

When f is a small constant, this gives a constant-factor approximation. Vertex cover is the special case where $U = E$, each set $S_v = \{e \in E : v \in e\}$ corresponds to a vertex, and every element has frequency exactly 2; the f-approximation recovers the factor of 2. When f is large, the greedy H_n-approximation from Lecture 1 is better.

We proved the greedy H_n-approximation combinatorially in Lecture 1. LP duality allows us to reinterpret that proof and determine the integrality gap of the set cover LP. Every LP has an associated *dual* program. For a primal LP in the form

$$\text{minimize } {}^t c\, x \text{ subject to } Ax \geq b,\ x \geq 0,$$

the dual is

$$\text{maximize } {}^t b\, y \text{ subject to } {}^t A\, y \leq c,\ y \geq 0.$$

The dual has one variable per primal constraint and one constraint per primal variable.

Theorem 3.4 (Weak duality). If x is primal feasible and y is dual feasible, then ${}^t c\, x \geq {}^t b\, y$.

Proof. Since ${}^t A\, y \leq c$ and $x \geq 0$, we have ${}^t c\, x \geq {}^t({}^t A\, y)\, x = {}^t y\, Ax$. Since $Ax \geq b$ and $y \geq 0$, we have ${}^t y\, Ax \geq {}^t y\, b = {}^t b\, y$. □

In other words, every dual feasible solution provides a lower bound on the primal optimum (for minimization problems). We do not need to solve the LP to use this: any feasible dual solution, however obtained, gives a certifiable lower bound.

The dual of the set cover LP (2) introduces a variable $y_e \geq 0$ for each element $e \in U$:

$$\begin{aligned} \text{maximize} \quad & \sum_{e \in U} y_e & \\ \text{subject to} \quad & \sum_{e \in S_j} y_e \leq c_j & \text{for all } j \\ & y_e \geq 0 & \text{for all } e \in U \end{aligned}$$

This is a *packing* problem: assign nonnegative values to elements so that no set is overloaded, and maximize the total value. By weak duality, any feasible packing gives a lower bound: $\sum_e y_e \leq \mathrm{OPT}$.

The greedy algorithm from Lecture 1 implicitly constructs a near-feasible dual solution. Recall that the greedy analysis assigns a price $\mathrm{price}(e)$ to each element e when it is first covered: if e is covered by a set S_j that covers k new elements at that step, then $\mathrm{price}(e) = c_j/k$. Setting $y_e = \mathrm{price}(e)$, the total cost of the greedy solution equals $\sum_{e \in U} y_e$. This y is not dual feasible in general: for a set S_j, we may have $\sum_{e \in S_j} y_e > c_j$. However, the greedy proof (Theorem 1.4) shows that $\sum_{e \in S_j} y_e \leq H_n \cdot c_j$ for every set S_j. Therefore, y/H_n is dual feasible, giving:

$$\frac{1}{H_n} \sum_{e \in U} y_e = \sum_{e \in U} \frac{y_e}{H_n} \leq \mathrm{OPT}$$

Hence $\mathrm{ALG} = \sum_e y_e \leq H_n \cdot \mathrm{OPT}_{\mathrm{LP}} \leq H_n \cdot \mathrm{OPT}$. The worst-case approximation ratio is the same as in Lecture 1, but we learn something new: the integrality gap of the set cover LP is at least H_n. Indeed, the dual solution has value ALG, so $\mathrm{OPT}_{\mathrm{LP}} \geq \mathrm{ALG}/H_n$ by weak duality, and there are instances where $\mathrm{ALG} = H_n \cdot \mathrm{OPT}$. Combined with the f-approximation from LP rounding (Theorem 3.3), this shows that the LP relaxation captures the approximability of set cover: the gap between the LP and integer optima is $\Theta(H_n)$ in the worst case, matching the best possible approximation ratio. This technique of building an approximately feasible dual solution and rescaling it is called *dual fitting*.

3.4 MAX-CUT

We now turn to a maximization problem. Given an undirected graph $G = (V, E)$ with nonnegative edge weights $w_{ij} \geq 0$, the MAX-CUT problem asks for a partition of V into two sets S and $\bar{S} = V \setminus S$ maximizing the total weight of edges crossing the cut: $w(S) = \sum_{\{i,j\} \in E,\, i \in S,\, j \notin S} w_{ij}$. MAX-CUT is NP-hard. Moreover, it is APX-hard: no PTAS exists unless P = NP.

A simple algorithm assigns each vertex to S independently with probability $1/2$.

Theorem 3.5. The randomized algorithm satisfies $\mathbb{E}[w(S)] \geq \frac{1}{2} w(E) \geq \frac{1}{2} \cdot \mathrm{OPT}$.

Proof. By linearity of expectation:

$$\mathbb{E}[w(S)] = \sum_{\{i,j\} \in E} w_{ij} \cdot \mathbb{P}[i \text{ and } j \text{ on different sides}] = \sum_{\{i,j\} \in E} w_{ij} \cdot \frac{1}{2} = \frac{w(E)}{2}.$$

Since $\mathrm{OPT} \leq w(E)$, we have $\mathbb{E}[w(S)] \geq \frac{1}{2} \cdot \mathrm{OPT}$. □

This can be derandomized by the method of conditional expectations: process vertices one by one, and place each on the side that maximizes the cut edges to previously placed vertices. This gives a deterministic $1/2$-approximation. Alternatively, any locally optimal cut (where no single vertex move improves the cut) has weight at least $w(E)/2$: local optimality means that each vertex has at least half its weighted degree on cut edges, and summing over all vertices gives $w(S) \geq w(E)/2$.

The $1/2$ barrier held for two decades. Can LP relaxation do better? We can formulate MAX-CUT as an ILP: for each edge $\{i, j\}$, introduce $y_{ij} \in \{0, 1\}$ indicating whether it is cut, and for each vertex i, introduce $x_i \in \{0, 1\}$ indicating its side. An edge is cut when its endpoints are on different sides:

$$\begin{aligned} \text{maximize} \quad & \sum_{\{i,j\} \in E} w_{ij}\, y_{ij} \\ \text{subject to} \quad & y_{ij} \leq x_i + x_j, \quad y_{ij} \leq 2 - x_i - x_j && \text{for all } \{i,j\} \in E \\ & x_i \in \{0,1\},\ y_{ij} \in \{0,1\} && \text{for all } i, \{i,j\} \end{aligned}$$

The LP relaxation replaces integrality with $0 \leq x_i, y_{ij} \leq 1$. Unfortunately, this relaxation is too weak: on any graph, setting $x_i = 1/2$ for all i and $y_{ij} = 1$ for all edges is feasible, with LP value $w(E)$. Since $\mathrm{OPT} \leq w(E)$, the LP always achieves $\mathrm{OPT}_{\mathrm{LP}} = w(E)$. On the complete graph K_n with unit weights, $\mathrm{OPT} \approx n^2/4$ while $w(E) = \binom{n}{2} \approx n^2/2$, so the integrality gap is $1/2$—no better than the trivial randomized algorithm. LP relaxation provides no advantage for MAX-CUT.

To break the $1/2$ barrier, Goemans and Williamson [31] used a stronger relaxation based on *semidefinite programming* (SDP). The starting point is a different integer formulation, better suited to relaxation. Instead of $x_i \in \{0, 1\}$, assign $z_i \in \{-1, +1\}$ to each vertex: $S = \{i : z_i = +1\}$. An edge $\{i, j\}$ is cut if and only if $z_i \neq z_j$, which happens when $z_i z_j = -1$. The objective becomes:

$$\text{maximize} \quad \sum_{\{i,j\} \in E} w_{ij} \cdot \frac{1 - z_i z_j}{2}$$

subject to $z_i \in \{-1, +1\}$ for all i.

The SDP relaxation replaces each scalar z_i with a unit vector $v_i \in \mathbb{R}^n$, and each product $z_i z_j$ with the

inner product $v_i \cdot v_j$. The constraint $z_i^2 = 1$ becomes $\|v_i\| = 1$. The relaxation is:

$$\begin{aligned} \text{maximize} \quad & \sum_{\{i,j\} \in E} w_{ij} \cdot \frac{1 - v_i \cdot v_j}{2} \\ \text{subject to} \quad & \|v_i\| = 1 \qquad \text{for all } i \in V \end{aligned} \tag{3}$$

This is a semidefinite program: the objective and constraints depend only on the pairwise inner products $v_i \cdot v_j$, so we can reformulate the problem in terms of the $n \times n$ *Gram matrix* Y defined by $Y_{ij} = v_i \cdot v_j$. The constraint $\|v_i\| = 1$ becomes $Y_{ii} = 1$, and a symmetric matrix Y with $Y_{ii} = 1$ arises as the Gram matrix of some unit vectors if and only if Y is positive semidefinite ($Y \succeq 0$). The SDP is therefore:

$$\begin{aligned} \text{maximize} \quad & \sum_{\{i,j\} \in E} w_{ij} \cdot \frac{1 - Y_{ij}}{2} \\ \text{subject to} \quad & Y_{ii} = 1 \qquad \text{for all } i \in V \\ & Y \succeq 0 \end{aligned}$$

This can be solved to arbitrary precision in polynomial time using interior point methods. To recover vectors for rounding, we compute a Cholesky-like factorization $Y^* = {}^tL\,L$; the rows of L give unit vectors v_i^* with $v_i^* \cdot v_j^* = Y_{ij}^*$.

Let $v_1^*, \dots, v_n^*$ be an optimal SDP solution with value $\mathrm{OPT}_{\mathrm{SDP}} \geq \mathrm{OPT}$. The rounding algorithm is beautifully simple: choose a random hyperplane through the origin and use it to partition the vertices.

Goemans–Williamson algorithm:

1. Solve the SDP relaxation (3) to obtain unit vectors $v_1^*, \dots, v_n^*$.
2. Choose a uniformly random unit vector r in the same space as the v_i^*.
3. Set $S = \{i \in V : v_i^* \cdot r \geq 0\}$.
4. Return the cut $(S, \bar{S})$.

The analysis rests on a geometric lemma: the probability that a random hyperplane separates two unit vectors equals the angle between them divided by π.

Lemma 3.1. For unit vectors $v_i, v_j \in \mathbb{R}^d$ and a uniformly random unit vector $r \in \mathbb{R}^d$,

$$\mathbb{P}[\operatorname{sign}(v_i \cdot r) \neq \operatorname{sign}(v_j \cdot r)] = \frac{\arccos(v_i \cdot v_j)}{\pi}.$$

Proof. The random hyperplane $\{x : r \cdot x = 0\}$ separates v_i and v_j if and only if it passes between them. Restricting to the two-dimensional plane spanned by v_i and v_j, the hyperplane intersects this plane in a random line through the origin. If the angle between v_i and v_j is $\theta = \arccos(v_i \cdot v_j)$, the line separates them if and only if it falls in one of the two arcs of angle θ (see Figure 3.1), which happens with probability $2\theta/(2\pi) = \theta/\pi$. □

Theorem 3.6 (Goemans–Williamson). The algorithm above achieves

$$\mathbb{E}[w(S)] \geq \alpha \cdot \mathrm{OPT}, \quad \text{where } \alpha = \min_{0 \leq \theta \leq \pi} \frac{2}{\pi} \cdot \frac{\theta}{1 - \cos\theta} \geq 0.878.$$

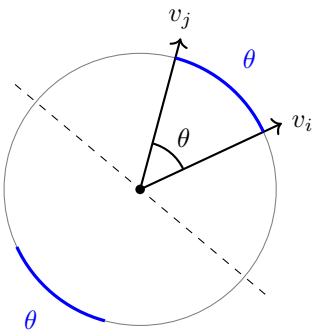

Figure 3.1: The random hyperplane (dashed line through the origin) separates v_i and v_j if and only if it passes through one of the two arcs of angle θ (shown in blue). This occurs with probability $2\theta/(2\pi) = \theta/\pi$.

Proof. By linearity of expectation and Lemma 3.1:

$$\mathbb{E}[w(S)] = \sum_{\{i,j\}\in E} w_{ij} \cdot \frac{\arccos(v_i^* \cdot v_j^*)}{\pi}$$

We compare this to the SDP value, which uses $(1 - v_i^* \cdot v_j^*)/2$ per edge. For each edge $\{i, j\}$, let $\theta_{ij} = \arccos(v_i^* \cdot v_j^*)$, so $v_i^* \cdot v_j^* = \cos\theta_{ij}$. We need to show $\frac{\theta}{\pi} \geq \alpha \cdot \frac{1-\cos\theta}{2}$ for all $\theta \in [0, \pi]$, i.e., $\frac{2\theta}{\pi(1-\cos\theta)} \geq \alpha$.

The function $g(\theta) = \frac{2\theta}{\pi(1-\cos\theta)}$ is continuous on $(0, \pi]$, with $g(\theta) \to \infty$ as $\theta \to 0^+$ and $g(\pi) = 1$. Since g is continuous and tends to $+\infty$ at 0, it attains its minimum α on $(0, \pi]$. Numerical computation gives $\alpha \approx 0.878$.

Therefore $\mathbb{E}[w(S)] \geq \alpha \sum_{\{i,j\}\in E} w_{ij} \cdot \frac{1-v_i^* \cdot v_j^*}{2} = \alpha \cdot \mathrm{OPT}_{\mathrm{SDP}} \geq \alpha \cdot \mathrm{OPT}$. □

The improvement from 0.5 to 0.878 comes entirely from the stronger relaxation: the SDP bound $\mathrm{OPT}_{\mathrm{SDP}}$ is much tighter than the LP bound $\mathrm{OPT}_{\mathrm{LP}} = w(E)$. The SDP relaxation has an integrality gap of exactly $\alpha \approx 0.878$.

The Goemans–Williamson result was a landmark in the theory of approximation algorithms, demonstrating that semidefinite programming can yield strictly better approximation ratios than linear programming. Assuming the Unique Games Conjecture [4], the ratio $\alpha \approx 0.878$ is the best possible for MAX-CUT in polynomial time, making the SDP relaxation optimally tight.

References

[1] David P. Williamson and David B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011.

[2] Irit Dinur and Shmuel Safra. On the hardness of approximating minimum vertex cover. *Annals of Mathematics*, 162(1):439–485, 2005.

[3] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4):634–652, 1998.

[4] Subhash Khot. On the power of unique 2-prover 1-round games. In *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*, pages 767–775, 2002.

[5] Subhash Khot and Oded Regev. Vertex cover might be hard to approximate to within $2-\varepsilon$. *Journal of Computer and System Sciences*, 74(3):335–349, 2008.

[6] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Technical Report 388, Graduate School of Industrial Administration, Carnegie Mellon University, 1976.

[7] Sartaj Sahni and Teofilo Gonzalez. P-complete approximation problems. *Journal of the ACM*, 23(3):555–565, 1976.

[8] Oscar H. Ibarra and Chul E. Kim. Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM*, 22(4):463–468, 1975.

[9] Ronald L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.

[10] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.

[11] Michel X. Goemans and David P. Williamson. New $\frac{3}{4}$-approximation algorithms for the maximum satisfiability problem. *SIAM Journal on Discrete Mathematics*, 7(4):656–666, 1994.

[12] Prabhakar Raghavan and Clark D. Thompson. Randomized rounding: A technique for provably good algorithms and algorithmic proofs. *Combinatorica*, 7(4):365–374, 1987.

[13] Teofilo F. Gonzalez. Clustering to minimize the maximum intercluster distance. *Theoretical Computer Science*, 38:293–306, 1985.

[14] Wen-Lian Hsu and George L. Nemhauser. Easy and hard bottleneck location problems. *Discrete Applied Mathematics*, 1(3):209–215, 1979.

[15] Dorit S. Hochbaum and David B. Shmoys. A unified approach to approximation algorithms for bottleneck problems. *Journal of the ACM*, 33(3):533–550, 1986.

[16] Vijay Arya, Naveen Garg, Rohit Khandekar, Adam Meyerson, Kamesh Munagala, and Vinayaka Pandit. Local search heuristics for k-median and facility location problems. *SIAM Journal on Computing*, 33(3):544–562, 2004.

[17] James H. Morris, Jr. and Vaughan R. Pratt. A linear pattern-matching algorithm. Technical Report 40, University of California, Berkeley, 1970.

[18] Donald E. Knuth, James H. Morris, Jr., and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977.

[19] Imre Simon. String matching algorithms and automata. In *Results and Trends in Theoretical Computer Science*, volume 812 of *Lecture Notes in Computer Science*, pages 386–395. Springer, Heidelberg, 1994.

[20] Robert S. Boyer and J Strother Moore. A fast string searching algorithm. *Communications of the ACM*, 20(10):762–772, 1977.

[21] R. Nigel Horspool. Practical fast searching in strings. *Software: Practice and Experience*, 10(6):501–506, 1980.

[22] Zvi Galil. On improving the worst case running time of the Boyer–Moore string matching algorithm. *Communications of the ACM*, 22(9):505–508, 1979.

[23] Richard Cole. Tight bounds on the complexity of the Boyer–Moore string matching algorithm. *SIAM Journal on Computing*, 23(5):1075–1091, 1994.

[24] Maxime Crochemore, Christophe Hancart, and Thierry Lecroq. *Algorithms on Strings*. Cambridge University Press, 2007.

[25] Maxime Crochemore and Dominique Perrin. Two-way string-matching. *Journal of the ACM*, 38(3):651–675, 1991.

[26] M. Lothaire. *Algebraic Combinatorics on Words*. Cambridge University Press, 2002.

[27] Anatoliy I. Serdyukov. On some extremal walks in graphs. *Upravlyaemye Sistemy*, 17:76–79, 1978.

[28] Anna R. Karlin, Nathan Klein, and Shayan Oveis Gharan. A (slightly) improved approximation algorithm for metric TSP. In *Proceedings of the 53rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 32–45, 2021.

[29] Dorit S. Hochbaum and David B. Shmoys. Using dual approximation algorithms for scheduling problems: Theoretical and practical results. *Journal of the ACM*, 34(1):144–162, 1987.

[30] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.

[31] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.